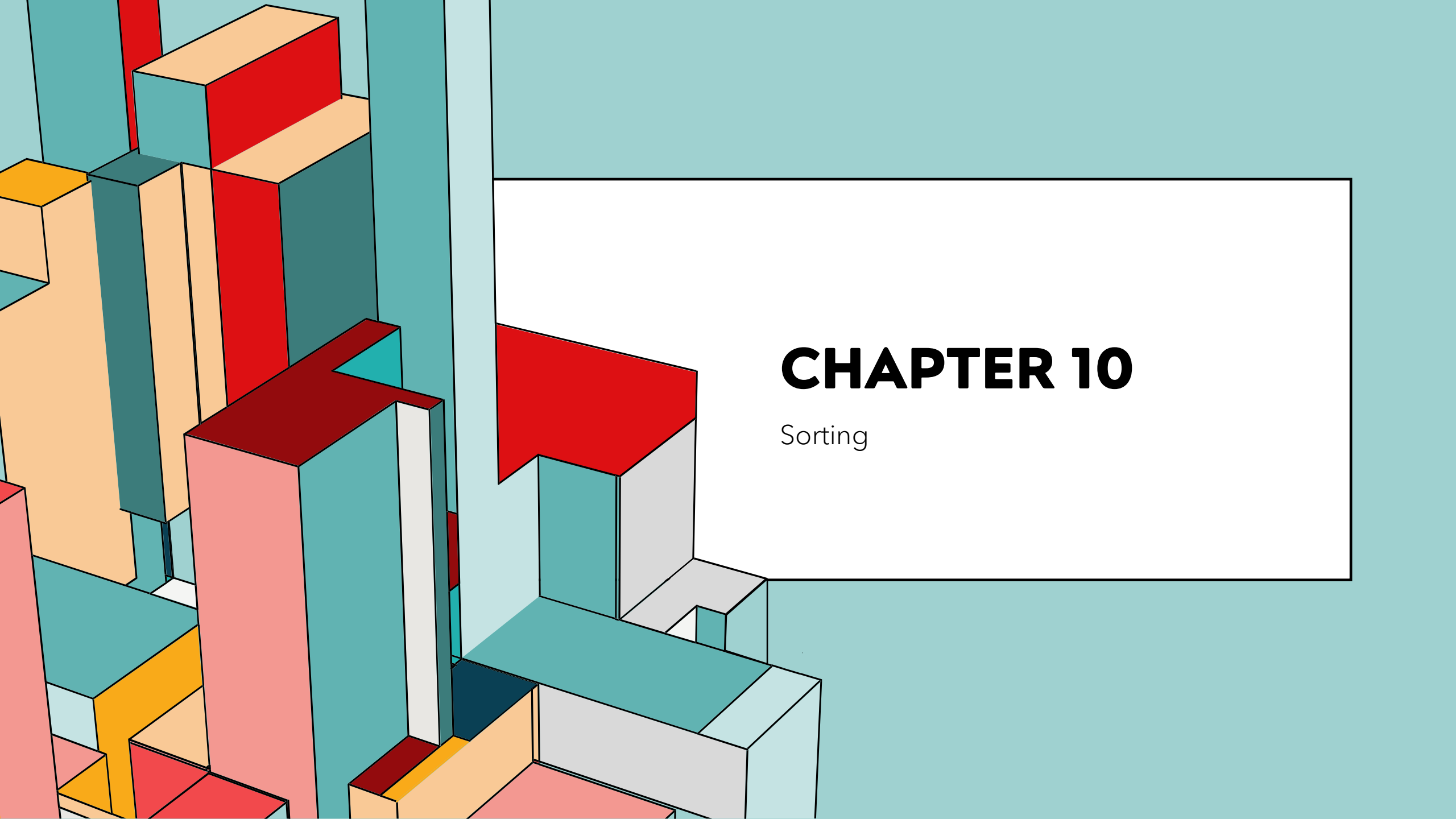




CHAPTER 10

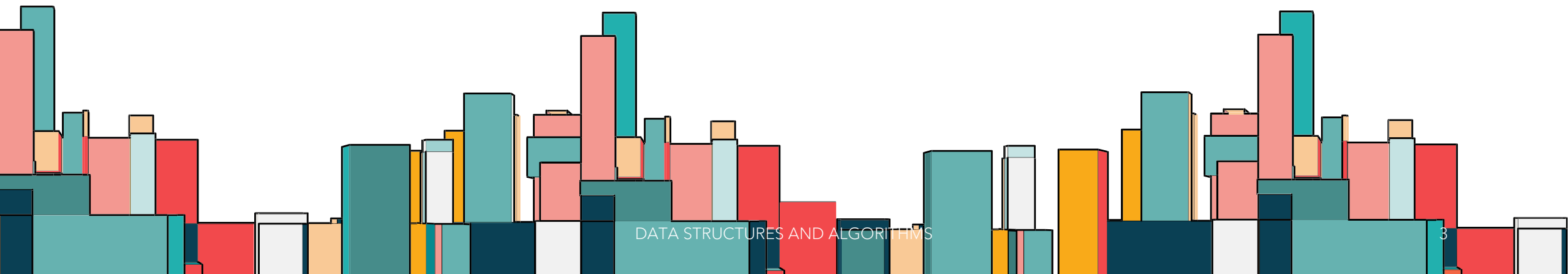
Sorting

SORTING

- ก่อนหน้านี้ ได้ศึกษาเรื่องอัลกอริทึมในการค้นหาข้อมูล (Searching) ทำให้ทราบว่า
 - หากมีการจัดเรียงข้อมูลเสียก่อน จะทำให้การค้นหามีประสิทธิภาพมากขึ้น
- การเรียงลำดับข้อมูล แบ่งเป็น
 - เรียงจากน้อยไปมาก เช่น -30 -15 0 15 30 หรือ 'A' 'B' 'C' 'D' เป็นต้น
 - เรียงจากมากไปน้อย เช่น 30 15 0 -15 -30 หรือ 'D' 'C' 'B' 'A' เป็นต้น
 - ซึ่งในบทเรียนจะเน้นที่ **การเรียงข้อมูลจากน้อยไปมาก**

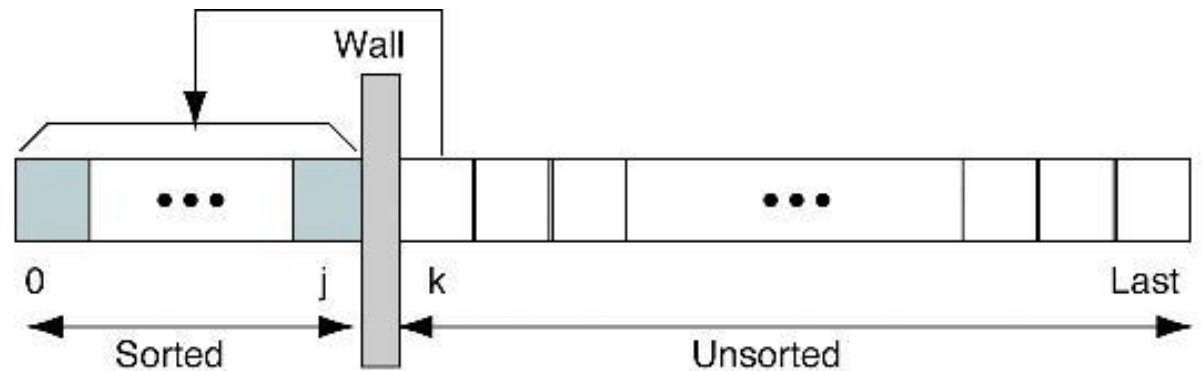
BASIC SORT ALGORITHM

- Insertion Sort
- Selection Sort
- Bubble Sort
- Merge Sort
- Quick Sort



INSERTION SORT

- ลิสต์ข้อมูลจะถูกแบ่งออกเป็น 2 ส่วน คือ
 - ลิสต์ย่อยที่ถูกจัดเรียงแล้ว (Sorted sublist)
 - ลิสต์ย่อยที่ยังไม่ถูกจัดเรียง (Unsorted sublist)
- หลักการ
 - จะนำข้อมูลตัวแรกของลิสต์ย่อยที่ยังไม่เรียง นำเข้าไปใส่ในลิสต์ย่อยที่มีการจัดเรียงอยู่แล้วในตำแหน่งที่เหมาะสม



INSERTION SORT ALGORITHM

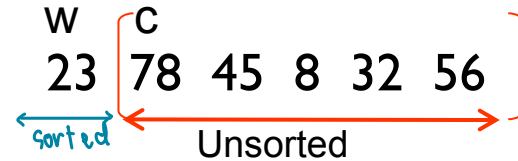
Algorithm insertionSort(list, last)

```
1  set current to 1
2  loop (until last element sorted)
    1  move current element to hold
    2  set walker to current-1
    3  loop (walker >= 0 AND hold key < walker key)
        1  move walker element right one element
        2  decrement walker
    4  end loop
    5  move hold to walker+1
    6  increment current
3  end loop
End insertionSort
```

INSERTION SORT EXAMPLE

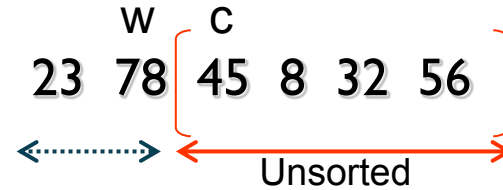
w : walker
c : current
hold = list[c]

Original list

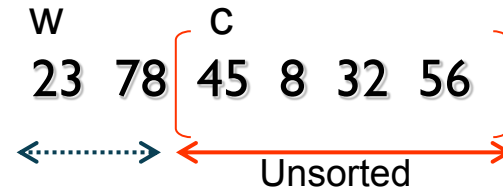


hold $\geq$ list[w]

After pass 1

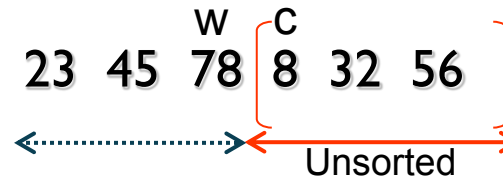


hold < list[w]

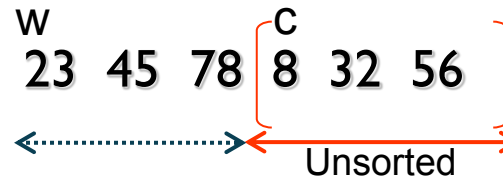


hold $\geq$ list[w]

After pass 2



hold < list[w]



w : walker
c : current
hold = list[c]

INSERTION SORT EXAMPLE

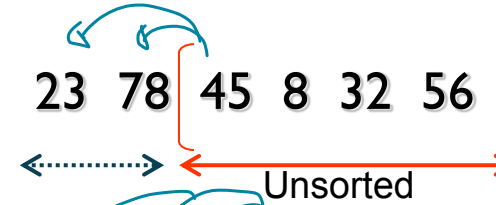
$O(n^2)$
 $n = 6$

Original list



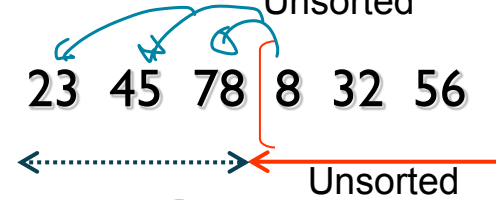
6 4 3 2 1 0
1 2 3 4 5 6

After pass 1



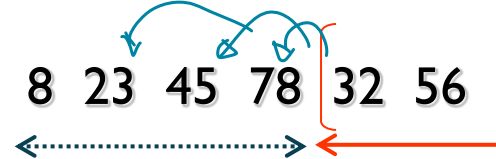
1 2

After pass 2



1 2 3

After pass 3



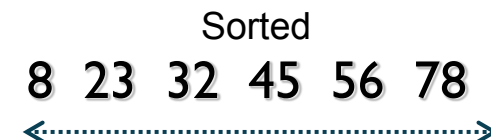
1 2 3

After pass 4



1 2

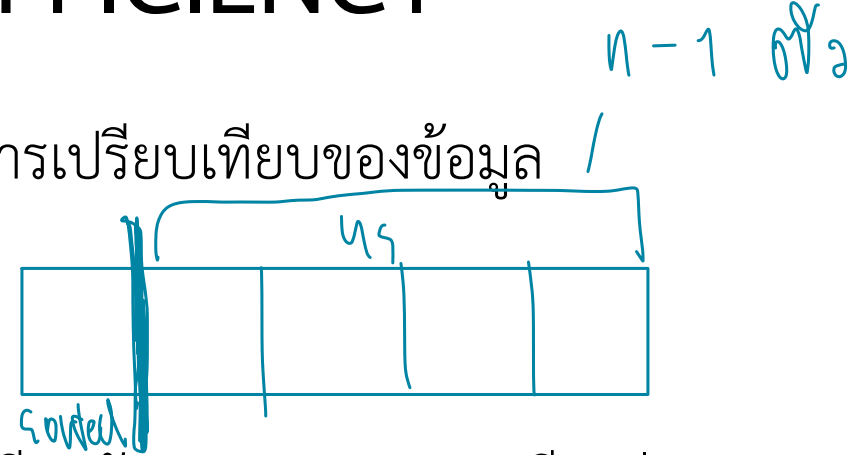
After pass 5



0 1 2 3 4 5

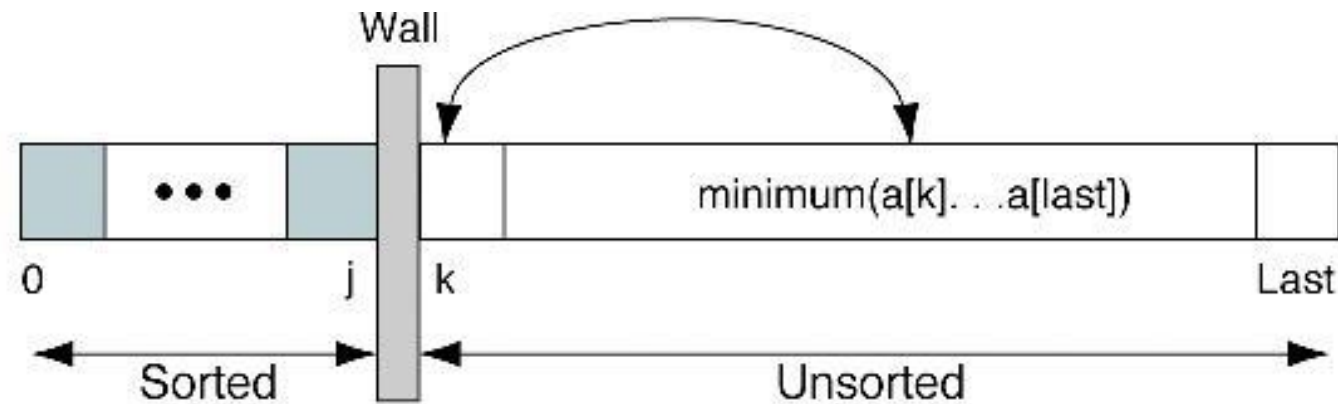
INSERTION SORT EFFICIENCY

- วัดประสิทธิภาพของวิธีเรียงข้อมูล โดยดูจำนวนการเปรียบเทียบของข้อมูล
- สังเกตว่า
 - ต้องเปรียบเทียบข้อมูลทั้งหมด $n-1$ ตัว
 - ซึ่งการเปรียบเทียบแต่ละตัว จะนำข้อมูลนั้นไปเทียบกับ Sorted Sublist อีกอย่าง มาก n ครั้ง (แต่ในที่นี้จะคิดในกรณีเฉลี่ยคือ $n/2$)
 - ดังนั้น จะได้ $f(n) = (n-1)(n/2)$
- การเรียงลำดับแบบ Insertion มีประสิทธิภาพคิดเป็น $O(n^2)$



SELECTION SORT

- หลักการ
 - เลือกค่าน้อยที่สุดจาก Unsorted sublist แล้วสลับที่ค่านั้นกับข้อมูลแรกของ Unsorted sublist แล้วจัดข้อมูลนั้นให้ไปอยู่ที่ Sorted sublist



SELECTION SORT ALGORITHM

Algorithm selectionSort(list, last)

1 set current to 0

→ หาตัวที่น้อยที่สุด

2 loop (until last element sorted)

1 set smallest to current

→ สมมติว่าตัวที่น้อยที่สุดคือ current

2 set walker to current + 1

3 loop (walker <= last)

→ เปรียบเทียบตัวที่น้อยที่สุดกับตัวที่น้อยกว่า

1 if (walker key < smallest key)

1 set smallest to walker

2 increment walker

4 end loop

→ smallest

5 exchange (current, smallest) *

6 increment current

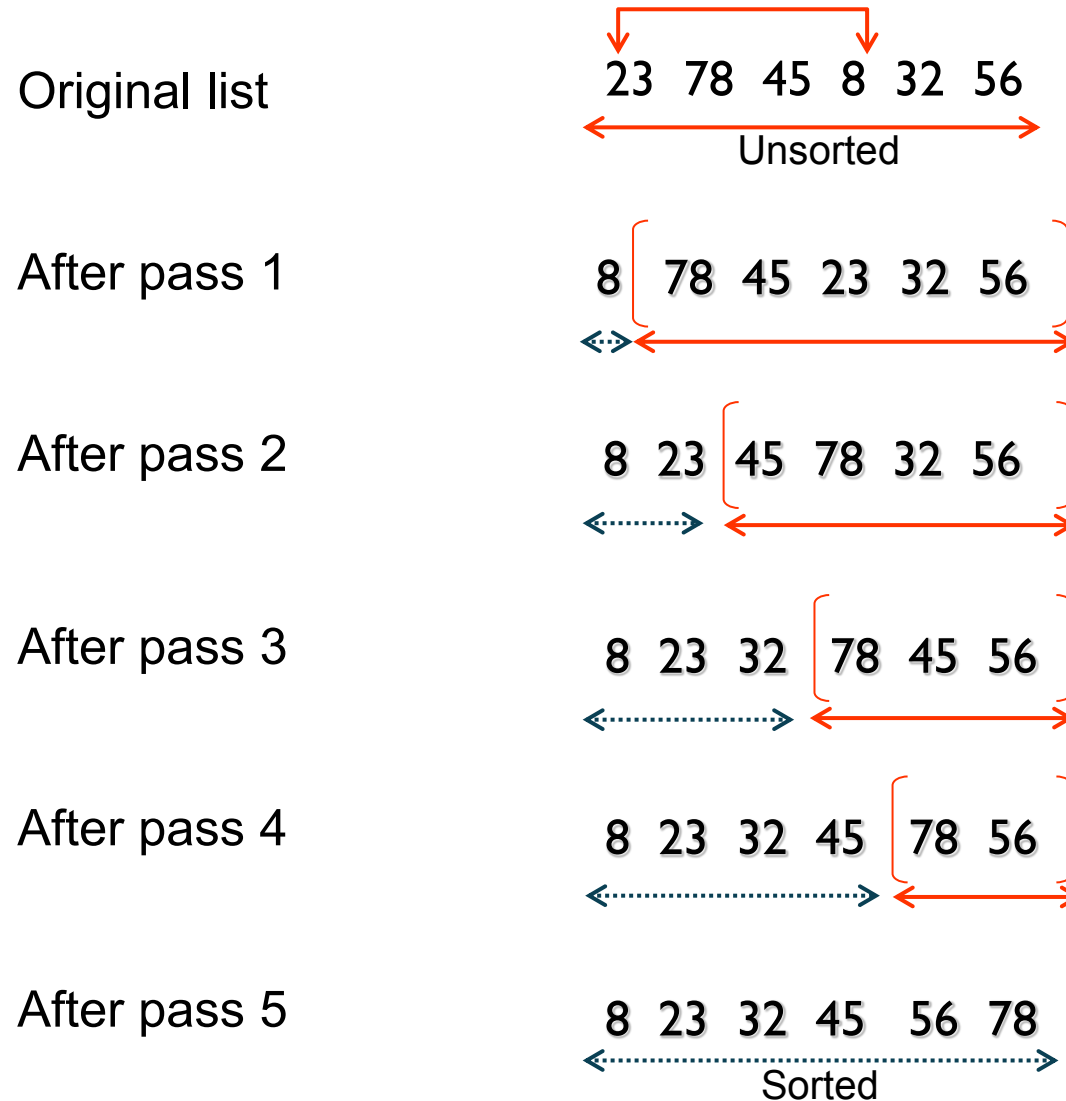
3 end loop

End selectionSort

w : walker
c : current
s : smallest



SELECTION SORT EXAMPLE



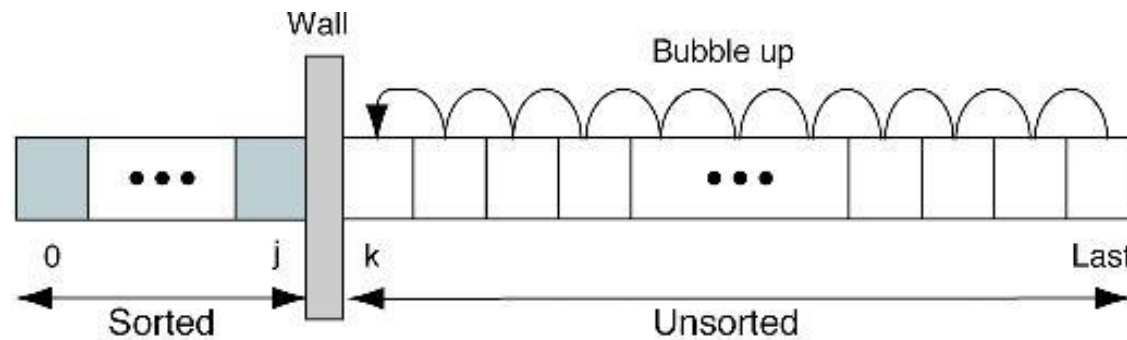
SELECTION SORT EFFICIENCY

- สังเกตว่า *walker* $\leftrightarrow$ *smallest*
 - ต้องทำการหาข้อมูลที่น้อยที่สุดใน Unsorted sublist ซึ่งจะมีจำนวนข้อมูลลดลงเรื่อยๆ ใน **การเปรียบเทียบ**แต่ละครั้ง คือ ในครั้งแรก Unsorted sublist จะมีข้อมูลทั้งหมด n ตัว ครั้งต่อไปจะเหลือ $n-1, n-2, \dots, 1$
 - เมื่อได้ค่าน้อยสุดก็สลับที่ข้อมูล (ในที่นี้ จะไม่นำมาคิดประสิทธิภาพ)
 - ดังนั้น จะได้ $f(n) = (n-1) + (n-2) + (n-3) + \dots + 1 = n(n-1)/2$
- การเรียงลำดับแบบ Selection มีประสิทธิภาพคิดเป็น $O(n^2)$

BUBBLE SORT

- หลักการ

- เริ่มพิจารณาเปรียบเทียบจากข้อมูลตัวท้ายสุดใน Unsorted sublist
- หากข้อมูลที่พิจารณามีค่าน้อยกว่าข้อมูลก่อนหน้า จะทำการสลับข้อมูลกัน
- ในทางตรงข้าม ก็จะเลือกพิจารณาข้อมูลก่อนหน้าแทน
- ทำจนหมด Unsorted sublist แล้วจึงย้ายข้อมูลหน้าสุด ไปอยู่ใน Sorted sublist
- นั่นคือ ข้อมูลที่มีค่าน้อยจะเริ่มขยับมาที่ส่วนหน้าของลิสต์ คล้ายฟองอากาศที่เบาจะลอยขึ้นมาเหนือน้ำ



BUBBLE SORT ALGORITHM

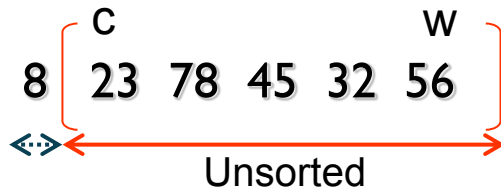
Algorithm bubbleSort(list, last)

```
1      set current to 0
2      set sorted to false
3      loop (current <= last AND sorted is false)
    1      set walker to last
    2      set sorted to true
    3      loop (walker > current)
        1      if (walker key < walker-1 key)
            1      set sorted to false
            2      exchange (walker, walker-1)
        2      end if
        3      decrement walker
    4      end loop
    5      increment current
4      end loop
End bubbleSort
```

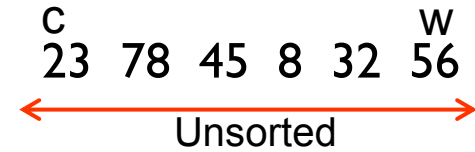
BUBBLE SORT EXAMPLE

```
w : walker
c : current
sorted = F
```

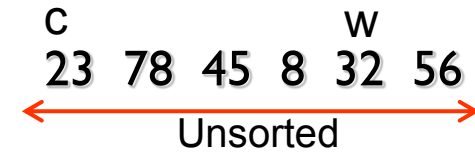
Original list



sorted = T



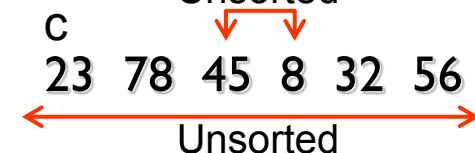
sorted = T



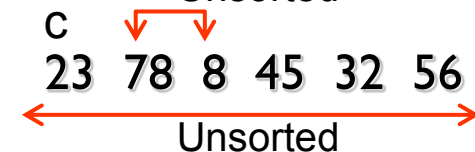
sorted = T



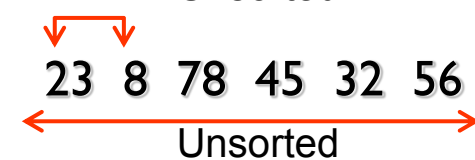
sorted = T



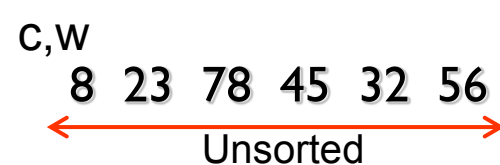
sorted = F



sorted = F

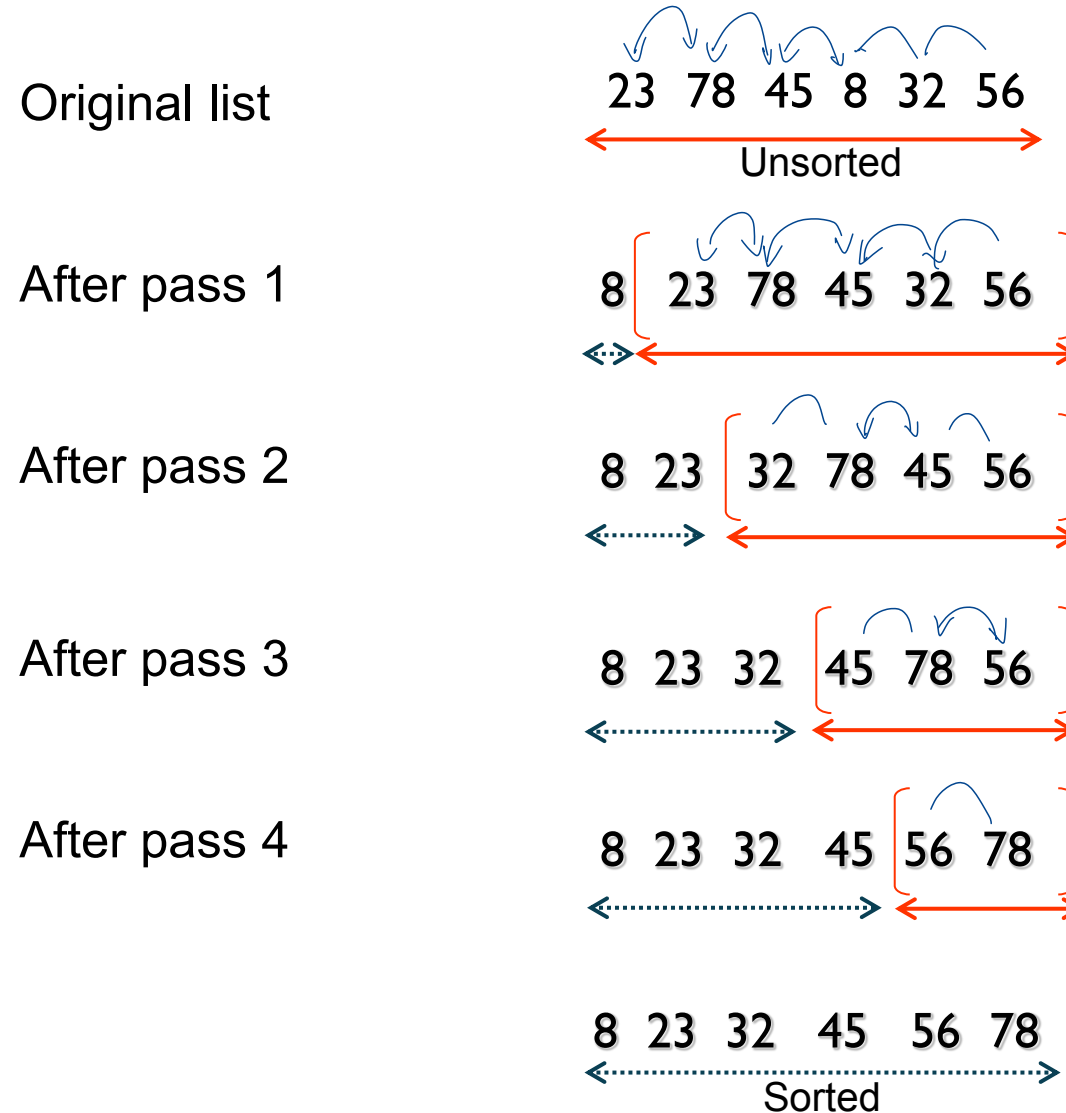


sorted = F



sorted = F

BUBBLE SORT EXAMPLE



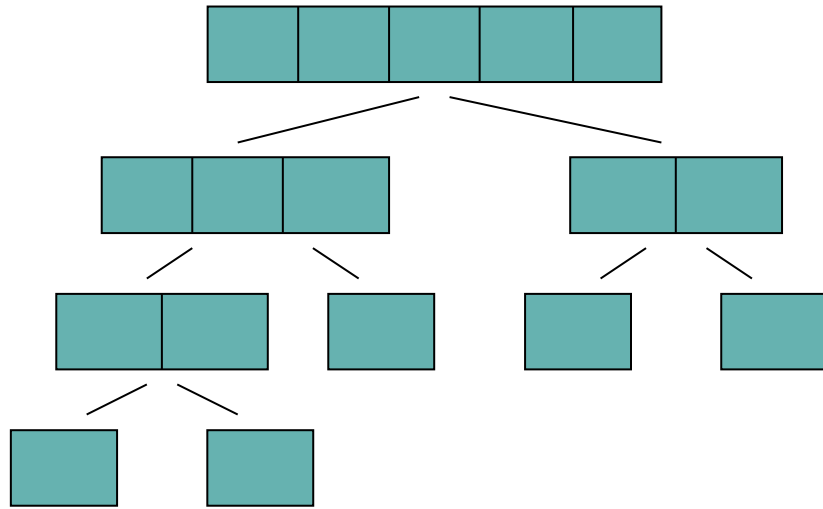
BUBBLE SORT EFFICIENCY

- สังเกตว่า
 - ต้องทำการเปรียบเทียบจากข้อมูลสุดท้ายใน Unsorted sublist เพื่อขยับข้อมูลที่มีค่าน้อยไปไว้ส่วนหน้าลิสต์ ซึ่งจำนวนของข้อมูลใน Unsorted sublist จะลดลงเรื่อยๆ ในการเปรียบเทียบแต่ละครั้ง
 - เมื่อได้ข้อมูลที่พิจารณามีค่าน้อยกว่าข้อมูลส่วนหน้า ก็สลับที่ข้อมูลนั้น (ในที่นี้ จะไม่นำมาคิดประสิทธิภาพ)
 - ดังนั้น จะได้ $f(n) = (n-1) + (n-2) + (n-3) + \dots + 1 = n(n-1)/2$
- การเรียงลำดับแบบ Bubble มีประสิทธิภาพคิดเป็น $O(n^2)$

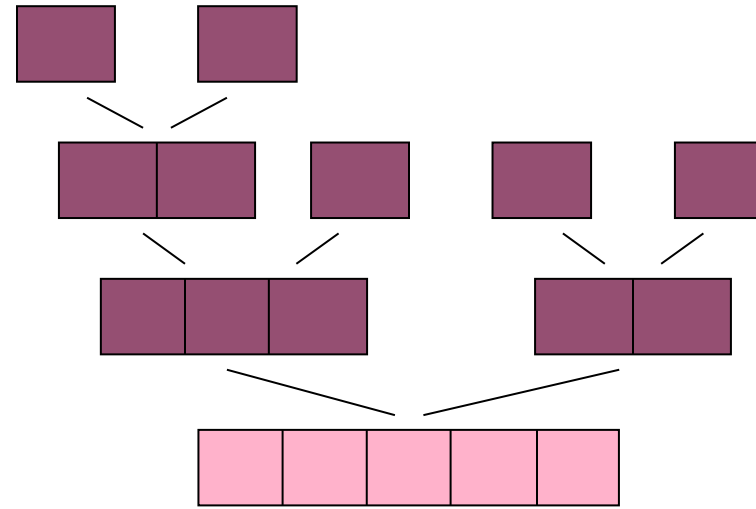
MERGE SORT

- หลักการ
 - ทำการแบ่งลิสต์ข้อมูลออกทีละ 2 ส่วน (Sublist) เท่าๆ กัน จนถึงระดับที่ไม่สามารถแบ่งได้อีก (Sublist มีข้อมูล 1 ตัว)
 - ทำการเรียงลำดับข้อมูลในแต่ละ Sublist
 - ทำการผสาน (Merge) Sublist ที่อยู่ติดกัน ให้เป็นลิสต์เดียว ไล่ขึ้นไปเรื่อยๆ จนเหลือลิสต์ข้อมูลเพียงลิสต์เดียว

MERGE SORT (CONT.)



Division phase



Merge phase

MERGE SORT ALGORITHM

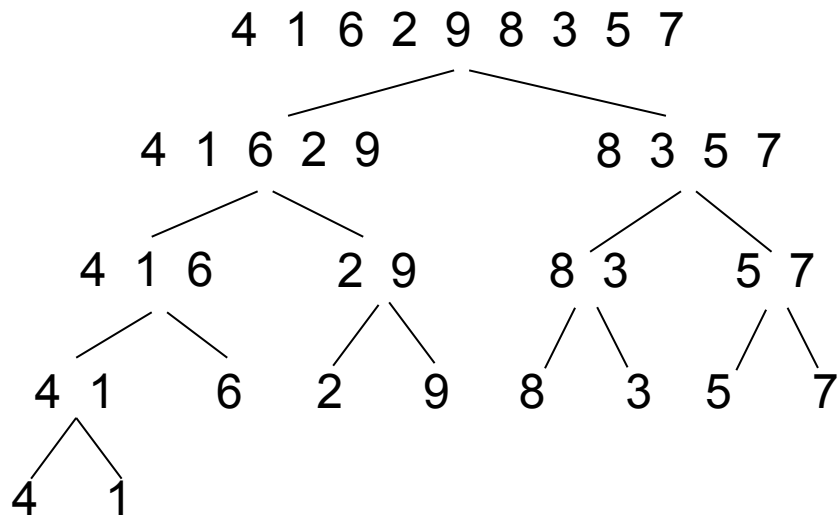
Algorithm mergeSort(list)

```
1  if length(list) ≤ 1
    1  return list
2  else
    1  middle = length(list) / 2
    2  divide list into two unsorted sublists,
       left and right, using middle
    3  left = mergeSort(left)
    4  right = mergeSort(right)
    5  result = merge(left, right)
    6  return result
3  end if
End mergeSort
```

Algorithm merge(left, right)

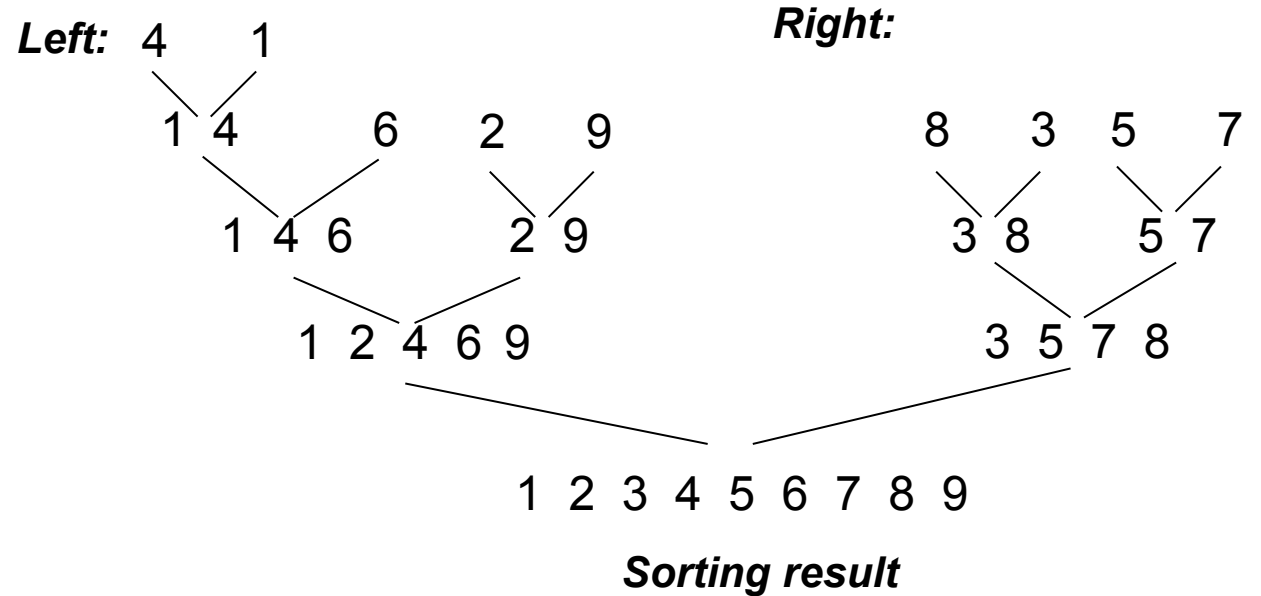
```
1  if length(left) = 0
    1  return right
2  if length(right) = 0
    1  return left
3  set zero to i and j
4  loop (i < length(left)) and (j < length(right))
    1  if (left[i] ≤ right[j])
        1  add left[i] to result
        2  increment i
    2  else
        1  add right[j] to result
        2  increment j
    3  end if
5  end loop
6  loop (i < length(left))
    1  add left[i] to result
7  end loop
8  loop (j < length(right))
    1  add right[j] to result
9  end loop
10 return result
End merge
```

1. Devide unsorted list into two sublists



Merge Sort Example

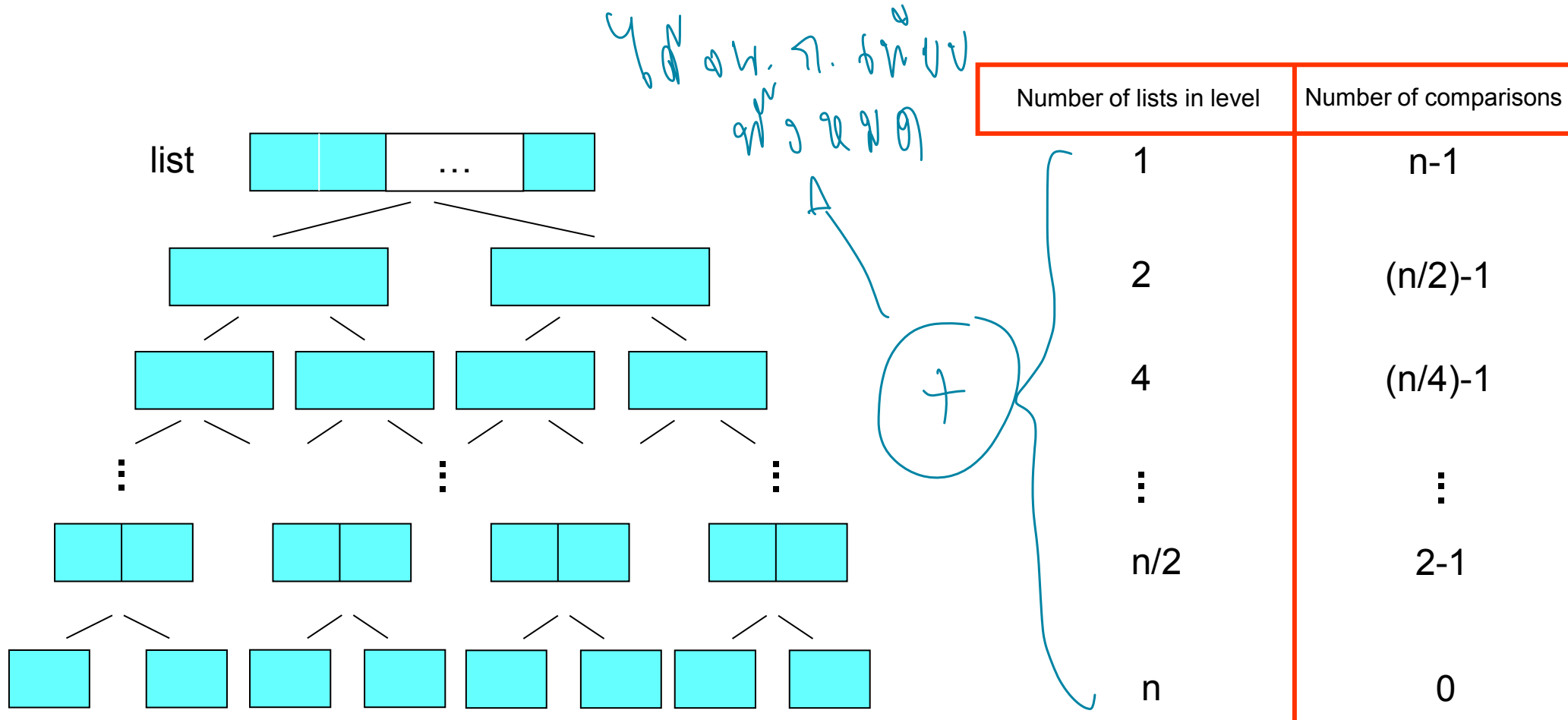
2. Merging and sorting



MERGE SORT EFFICIENCY

- สังเกตว่า
 - ประสิทธิภาพของวิธีนี้ จะเกิดจากผลรวมของการเปรียบเทียบข้อมูลในแต่ละ Sublist จนครบทั้งหมด
 - ซึ่งการเปรียบเทียบในแต่ละ Sublist จะเกิดขึ้นเมื่อต้องการผสาน Sublist ที่อยู่ใกล้เคียงกัน
 - นั่นคือ ถ้ามี Sublist 2 ชุด ซึ่งต่างมีจำนวนข้อมูล $N/2$ (เมื่อรวมแล้วจะได้ Sublist ที่มีจำนวนข้อมูล N ตัว) จะมีการเปรียบเทียบมากที่สุด $N-1$ ครั้ง

MERGE SORT EFFICIENCY (CONT.)



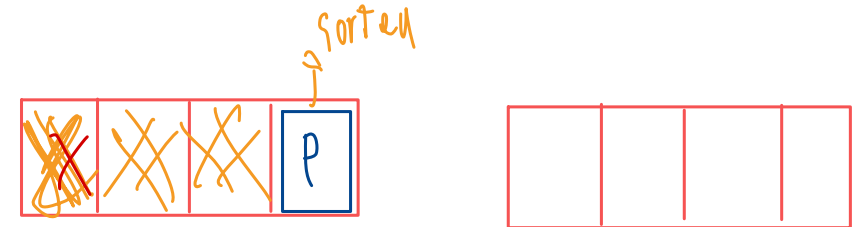
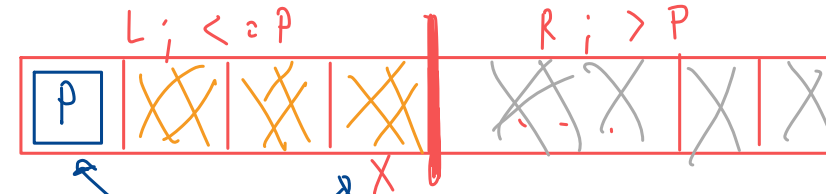
MERGE SORT EFFICIENCY (CONT.)

$$\begin{aligned}\text{Total times} &= \sum_{i=1}^N (\text{Number of lists in level } i) \times (\text{Number of comparisons in level } i) \\&= 1 \times (n-1) + 2 \times ((n/2)-1) + 4 \times ((n/4)-1) + \dots + (2-1) \times (n/2) \\&= n-1 + n-2 + n-4 + \dots + n - n/2 \\&= \boxed{n + n + n + \dots + n} - \boxed{(1 + 2 + 4 + 8 + \dots + n/2)} \\&\quad \text{n times = level of tree} \\&= n \log_2 n - (n - 1)\end{aligned}$$

The merge sort efficiency is $O(n \log_2 n)$

QUICK SORT

- หลักการ
 - ใช้ค่า Pivot แบ่งลิสต์ข้อมูลออกเป็น 2 ชุด ซึ่งลิสต์ที่ถูกแบ่งไม่จำเป็นต้องมีขนาดเท่ากัน
 - Pivot เป็นค่าที่ตำแหน่งแรกของลิสต์ที่ต้องการแบ่ง
 - ลิสต์ข้อมูลฝั่งซ้ายจะต้องมีค่าน้อยกว่าหรือเท่ากับ Pivot
 - ลิสต์ข้อมูลฝั่งขวาจะต้องมีค่ามากกว่า Pivot
 - เมื่อแบ่งแล้ว ให้สลับ Pivot กับค่าที่ตำแหน่งสุดท้ายของ Sublist ฝั่งซ้าย
 - กำหนด Pivot ใหม่ แล้วทำเหมือนเดิมไปเรื่อยๆ จนไม่สามารถแบ่งลิสต์ได้อีก

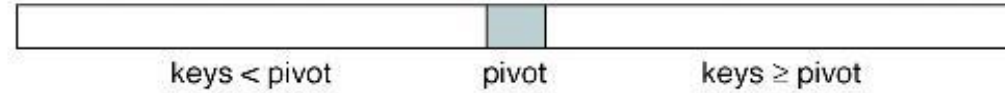


QUICK SORT (CONT.)

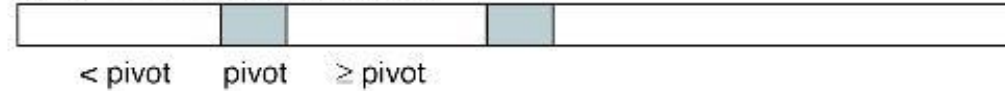
- การแบ่งลิสต์เป็น 2 ชุด ตามค่า Pivot มีขั้นตอนดังนี้
 - ใช้ first และ last มาช่วยแบ่ง โดยกำหนด
 - First เริ่มต้นชี้ที่ตำแหน่งข้อมูลที่สอง (ถัดจากค่า Pivot)
 - Last เริ่มต้นชี้ที่ตำแหน่งสุดท้ายของลิสต์
 - เริ่มต้น พิจารณาค่าที่ first ชี้อยู่
 - ถ้าค่าข้อมูลน้อยกว่าหรือเท่ากับ Pivot ให้ย้าย first ชี้อข้อมูลถัดไป (ทางขวา)
 - ถ้าค่าข้อมูลมากกว่า Pivot ให้ไปพิจารณาข้อมูลที่ last ชี้อยู่
 - ถ้าข้อมูลที่ last ชี มีค่าน้อยกว่าหรือเท่ากับ Pivot ให้ทำการสลับที่ข้อมูลที่ first และ last ชี้อัน
 - ถ้าไม่ใช่ ให้ย้าย last ชี้อข้อมูลก่อนหน้า (ไล่ไปทางซ้าย)
 - ทำไปเรื่อยๆ จนกว่า last จะอยู่ด้านหน้าของ first และใช้ Last แบ่งลิสต์

QUICK SORT (CONT.)

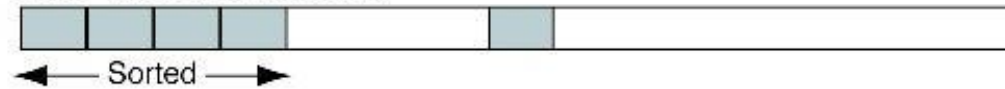
After first partitioning



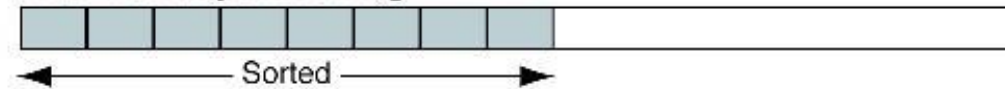
After second partitioning



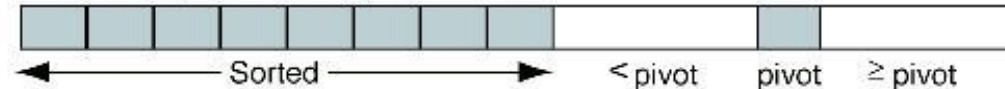
After third partitioning



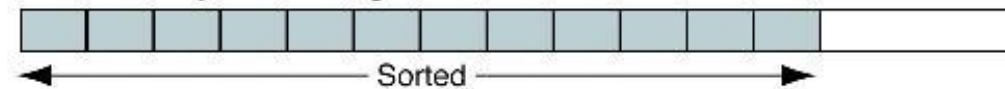
After fourth partitioning



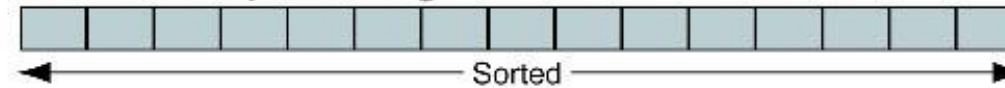
After fifth partitioning



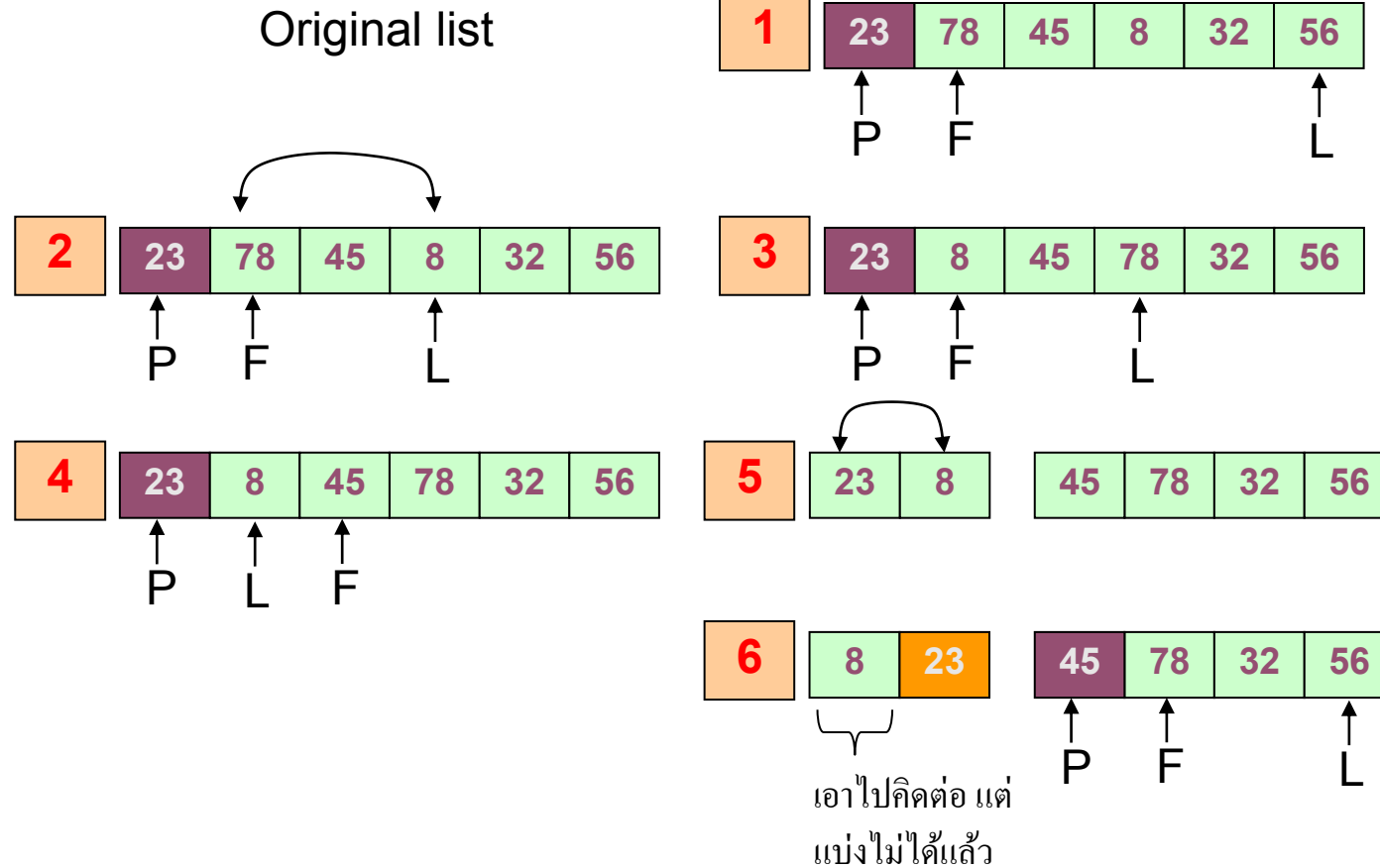
After sixth partitioning



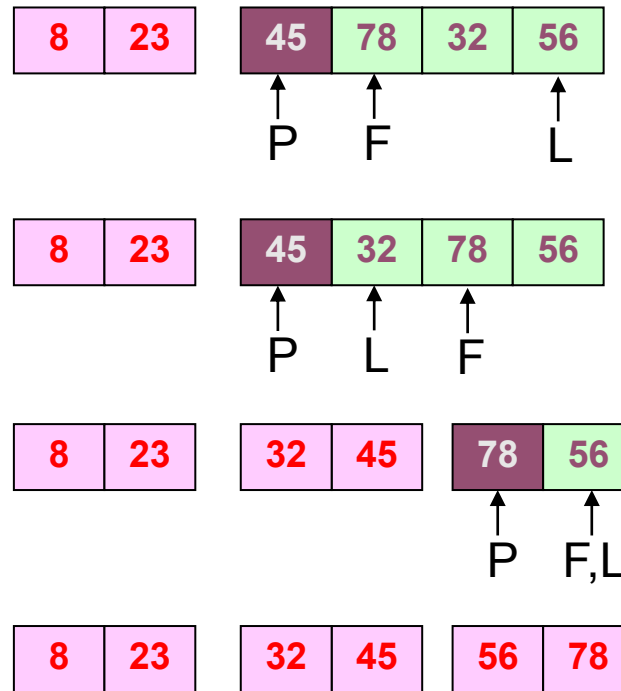
After seventh partitioning



QUICK SORT EXAMPLE



QUICK SORT EXAMPLE (CONT.)



QUICK SORT EFFICIENCY

- สังเกตว่าจะคล้ายกับวิธี Merge Sort
 - วัดจาก ผลรวมของการเปรียบเทียบข้อมูลในแต่ละ Sublist จนครบทั้งหมด
 - ซึ่งการเปรียบเทียบในแต่ละ Sublist จะเกิดขึ้นเมื่อต้องการแบ่ง Sublist ออกเป็น 2 ชุด ซึ่งถ้า Sublist มี N ข้อมูล ก็จะมีการเปรียบเทียบประมาณ N ครั้ง
 - จะเห็นว่า ประสิทธิภาพของวิธีนี้ ขึ้นอยู่กับค่า Pivot
 - ถ้าค่า Pivot สามารถแบ่งลิสต์ออกเป็น 2 ชุดเท่าๆ กันได้ จะเป็นกรณีที่ดีที่สุด นั่นคือเป็น $O(N \log_2 N)$
 - กรณีที่ลิสต์ที่จะแบ่ง เกือบจะเรียงลำดับสมบูรณ์ จะมีประสิทธิภาพต่ำ ซึ่งวัดได้ $O(N^2)$

EXERCISE

ให้จัดเรียงชุดข้อมูลต่อไปนี้

503 87 512 61 908 170 897

ด้วยอัลกอริทึมต่อไปนี้ (ให้เขียนเป็นลำดับขั้นตอน พอเข้าใจ)

- Insertion Sort
- Selection Sort
- Bubble Sort
- Merge Sort
- Quick Sort